Yocto Custom Embedded Linux Project Guide

Target Architecture: QEMU x86-64 | **Release:** Poky Scarthgap 5.0 LTS | **Kernel:** Linux 6.6.151

Author: Bharath G Devadiga | **Repository:** github.com/BharathGDevadiga/yocto-custom-image

1. Executive Summary & Architecture

This project demonstrates the complete end-to-end design, implementation, and cross-compilation of a custom embedded Linux distribution using the Yocto Project (Poky Scarthgap). The system features an out-of-tree Linux 6.6 character device driver, an embedded telemetry daemon with checksum verification, and a POSIX socket networking suite.

Component	Package / Module	Key Role & Architecture
Kernel Driver	`sensor_kmod` (6.6)	Exposes `/dev/bharath_sensor`; thread-safe mutex locking, ioctl configuration, and atomic telemetry streaming.
User-space App	`sensor-simulator`	Reads binary struct from `/dev/bharath_sensor`, tests ioctl calibration, and computes Fletcher-16 checksums.
Network Suite	`netcomm`	POSIX TCP/UDP client-server binaries verifying Linux socket IPC in QEMU.
Custom Layer	`meta-bharath`	Contains all layer metadata, kernel recipe, app recipes, and image definition.
OS Image	`bharath-image`	Minimal production root filesystem (97 MB) with Dropbear SSH and auto-loaded modules.

2. Step-by-Step Implementation

Step 2.1: Authoring the Linux 6.6 Character Device Driver

The kernel module (`sensor_kmod.c`) registers a dynamic major number with `alloc_chrdev_region()` and uses modern Linux 6.4+ API `class_create("bharath_class")` to automatically instantiate `/dev/bharath_sensor` via `eudev` .

Concurrency & Mutex Locking: To guarantee thread safety across concurrent readers, critical sections are guarded by `mutex_lock_interruptible(&sensor_mutex)` , safely releasing before `copy_to_user()` to prevent holding locks across potential page faults.

IOCTL Interface: Supports `_IO` , `_IOR` , and `_IOW` ioctls to reset driver telemetry statistics, read total frames served, and apply runtime hardware temperature calibration offsets.

When user space reads the device node, the driver packs hardware telemetry and safely transfers it to user space via `copy_to_user()` :

```
struct bharath_sensor_data {
    u64 timestamp_jiffies; /* get_jiffies_64() */
    s32 temperature_mC;    /* 20000 mC to 35000 mC + ioctl calibration */
    u32 pressure_Pa;       /* 98000 Pa to 104000 Pa */
} __attribute__((packed));
```

Step 2.2: User-Space Telemetry Client with Fletcher-16 Checksum

The application (`sensor-simulator.c`) opens `/dev/bharath_sensor` with `O_RDONLY` and receives the binary payload directly from kernel memory. It applies a 16-bit Fletcher checksum across the struct buffer to guarantee data integrity before outputting formatted telemetry frames.

Step 2.3: BitBake Recipe Architecture

- `recipes-kernel/sensor-kmod/sensor-kmod_1.0.bb` : Inherits `module` and sets `KERNEL_MODULE_AUTOLOAD += "sensor_kmod"` to load the driver at boot automatically.
- `recipes-apps/sensor-simulator/sensor-simulator_1.0.bb` : Cross-compiles the C binary with `${CC} ${CFLAGS} ${LDFLAGS}` into `/usr/bin/` .
- `recipes-core/images/bharath-image.bb` : Inherits `core-image` and appends `sensor-simulator` , `netcomm` , and `sensor-kmod` to `IMAGE_INSTALL` .

3. Running & Verifying in QEMU

Method A: Instant Standalone Launch (No Yocto / Poky Required)

If running from the pre-built GitHub Release artifacts (`bzImage` and rootfs `.ext4`):

```
# On Linux or WSL:
chmod +x run-prebuilt.sh
./run-prebuilt.sh

# On Windows:
Double-click run-prebuilt-windows.bat
```

Method B: Launching via Poky Build Environment

To boot the compiled image headlessly from the full Yocto build directory:

```
cd ~/poky
source oe-init-build-env build
runqemu nographic
```

Log in at the prompt:

```
qemux86-64 login: root
(Press Enter — no password required)
```

Live Verification Commands Inside QEMU

```
# 1. Verify kernel module is auto-loaded:
lsmod | grep sensor_kmod
# Output: sensor_kmod   12288  0

# 2. Verify character device node created by eudev:
ls -l /dev/bharath_sensor
# Output: crw-rw-rw- 1 root root 246, 0 /dev/bharath_sensor

# 3. Stream 5 live frames with checksum validation:
sensor-simulator -n 5
```

Verified Live Output Captured in QEMU:

```
[2026-11-01 13:27:34] [JIFFIES=4294739672] KERNEL_SENSOR: TEMP=22.07°C | PRESS=1004.17 hPa | CHECKSUM=0xBAB9 (VALID)
[2026-11-01 13:27:36] [JIFFIES=4294741707] KERNEL_SENSOR: TEMP=28.16°C | PRESS=1025.39 hPa | CHECKSUM=0x04E3 (VALID)
```

4. The Proper Command to Exit QEMU

No Keyboard Shortcuts Needed — Use the Standard Power Command:

While inside QEMU at the `root@qemux86-64:~#` prompt, simply type:

```
poweroff
```

This initiates an ACPI shutdown sequence: services are stopped, disk filesystems unmount cleanly, and QEMU terminates, returning you immediately to your Ubuntu terminal prompt.

5. How to Close WSL Completely

Follow these two clean steps to shut down WSL when you are finished:

1. **Close the Ubuntu Terminal:**

   In your Ubuntu terminal prompt, simply type:

   ```
   exit
   ```

2. **Shut Down WSL Engine from Windows PowerShell:**

   Open Windows PowerShell and run:

   ```
   wsl --shutdown
   ```

   This completely stops all WSL virtual machine processes and releases all memory back to Windows.

6. Adapting for Real Hardware & Physical Sensors

This project is architected as an extensible BSP framework. To adapt this system from QEMU simulation to physical silicon (e.g., Raspberry Pi, BeagleBone, NXP i.MX, or STM32MP1):

A. In the Kernel Driver (`sensor_kmod.c`)

Open `meta-bharath/recipes-kernel/sensor-kmod/files/sensor_kmod.c` and modify `sensor_read()` :

```
// 1. For I2C Sensors (e.g. BMP280, BME680, MPU6050):
#include <linux/i2c.h>
// Replace random entropy with I2C bus transactions:
s32 raw_temp = i2c_smbus_read_word_data(client, BMP280_TEMP_REG);
data.temperature_mC = bmp280_compensate_temperature(raw_temp);

// 2. For SPI Sensors:
#include <linux/spi/spi.h>
spi_sync_transfer(spi_device, &transfer, 1);
```

B. Device Tree Binding (`.dts`)

In physical hardware, bind your driver to the board's Device Tree:

```
&i2c1 {
    status = "okay";
    bharath_sensor@76 {
        compatible = "bharath,sensor-v1";
        reg = <0x76>;
    };
};
```

C. User-Space Extensions (`sensor-simulator.c`)

Pipe telemetry from `/dev/bharath_sensor` into:

- **Automotive CAN Bus:** Forward frames to `can0` via Linux `SocketCAN` .
- **Cloud IoT:** Publish JSON telemetry payloads to MQTT brokers (AWS IoT Core / ThingsBoard).
- **IPC Networking:** Stream over TCP/UDP sockets using the bundled `netcomm` package.

7. Official Yocto Lifecycle & `devtool` Workflow

BitBake processes every recipe through a deterministic task pipeline:

`do_fetch` → `do_unpack` → `do_patch` → `do_configure` → `do_compile` → `do_install` → `do_package` → `do_rootfs`

Iterative Development with `devtool`

Instead of rebuilding the entire image when editing code, use the official Yocto `devtool` workflow:

```
# 1. Extract recipe source into an active workspace:
devtool modify sensor-simulator

# 2. Fast incremental compilation (takes 2 seconds):
devtool build sensor-simulator

# 3. Live deployment to running QEMU target over network:
devtool deploy-target sensor-simulator root@192.168.7.2

# 4. Commit verified changes back into the recipe as a patch:
devtool finish sensor-simulator meta-bharath
```

8. Reclaiming 50+ GB on Your Laptop (Optional)

Once your release binaries are backed up in `release-artifacts/`, you can reclaim your disk space:

1. **Clean Build Cache in Ubuntu:**

   ```
   rm -rf ~/poky/build/tmp ~/poky/build/sstate-cache ~/poky/build/downloads
   sudo fstrim -av
   ```

2. **Compact Virtual Disk in PowerShell (as Administrator):**

   ```
   wsl --shutdown
   diskpart
   ```

   Inside `DISKPART>`:

   ```
   select vdisk file="<path-to-your-wsl-ext4.vhdx>"
   attach vdisk readonly
   compact vdisk
   detach vdisk
   exit
   ```